

Trabalho 2

Arquitetura de computadores

Vinícius de Jesus Carmo

O programa que eu criei não é nada complexo. Não é nada de outro mundo. Eu procurei satisfazer os requisitos mencionados na especificação do trabalho invés de fazer um programa complicado.

```
1  #include <stdio.h>
2
3  int somatorio_N(int N){
4      if(N == 1) return 1;
5      else return N + somatorio_N(N - 1);
6  }
7
8  int main(){
9      int N;
10     scanf("%d", &N);
11     int j = somatorio_N(N);
12     printf("%d\n", j);
13     for(int i = j; i >= 0; i--){
14         printf("%d\n", i);
15     }
16 }
17
18
```

O programa traduzido foi este ao lado. O código é simples e é uma função recursiva que recebe um inteiro N e retorna o somatório de 1 até N junto com a impressão de todos os números de N até 0. Até os \n dos prints foram introduzidos na impressão. É simples mas contém desvio condicional, repetição, funções, recursividade e sequenciamento. 5 dos 6 pedidos no trabalho.

```
4  .text
5  .globl main #Defino que o main é um escopo global
6
7  main:
8      li $v0, 5 #Gravo o valor 5 no $v0, e chamo o syscall que indica que o sistema irá ler um inteiro.
9      syscall #Chamada do sistema
10     move $a0, $v0 #Movo do $v0 para o reg. $a0. Isso ocorre pq usamos o v0 para saídas e o a0 para entrada de dados.
11     jal somatorio #Faço um 'Jump and Link', ou seja, ele pula para somatorio e registra o endereço de memória de PC + 4 num registrador chamado $ra
12     move $a0, $v0 #Após retornar do Jump and Link ele move o valor retornada da função que está guardado em $v0 para $a0
13     li $v0, 1 #Apenas para poder gravar o valor 1(significa "Imprima um inteiro") no $v0 sem perder o dado que seria sobreescrito
14     syscall #chamo o sistema, ele imprime o valor que está em $a0
15     move $t0, $a0 #movo para registrador temporario, explicarei abaixo o motivo
16     li $v0, 11 #Gravo em v0 o codigo de impressão de um caracter
17     li $a0, 10 #Gravo em a0 o codigo correspondente a o \n na tabela ascii
18     syscall #Sistema imprime o que está em a0 e como o cdoigo em v0 é para caracter ele imprime o codigo da tabela ascii
19     #Por esse motivo que eu passo o valor de a0 para t0, para não ser sobreescrito
20     jal imprimir_numeros #Chamo a função que implementa um for para imprimir todos os números de N até 0
21     li $v0, 10 #Gravo o valor 10(Que significa "Encerre o programa") no $v0
22     syscall #chamo o sistema onde ver o valor 10 e encerra o programa
23
```

Nos comentários já está bastante detalhado então não irei ser redundante aqui nos slides.

O main lê o número de entrada, e chama a função recursiva. Ao voltar continua e prepara o sistema para pular para o for de impressão dos números. E por fim encerra o programa.

```
somatorio:
    beq $a0, 1, retorno #"Branch if Equal", se o valor no $a0 for == 1 então pule para retorno

# Prólogo da recursão, onde a gente salva o endereço de memória de retorno atual e o valor atual.
    addi $sp, $sp, -8 #Crio espaço na pilha fazendo um $sp receber o valor que já está nele -8. Então criei 8 espaços a mais.
    sw $ra, 4($sp) #guardo no endereço de memoria de $sp + 4 o valor que está em $ra, ou seja, o endereço de retorno atual
    sw $a0, 0($sp) #e guardo em $sp + 0 o valor atual da recursão que está em $a0

#Depois que eu preparei o sistema e suas variáveis, posso chamar a recursão sem sobreescrever os dados.
#Em C o programa está na parte de: return N + somatorio(N - 1)
#Então eu preparo esse N - 1 para ser jogado na recursão

# A recursão em si vem nessa parte
    addi $a0, $a0, -1 #Somo addi(soma de um imediato com sinal. Se fosse add ele esperava soma entre registradores) com -1
    jal somatorio #agora que o valor atual foi decrementado ou seja, N - 1, chamo a função recursivamente.

    lw $a0, 0($sp) #Retornando da recursão, recuperamos o valor que foi guardado anteriormente e guardamos em $a0
    lw $ra, 4($sp) #E o mesmo para o endereço de retorno do anterior
    addi $sp, $sp, 8 #E removemos o espaço que foi separado na pilha para guardar esses valores

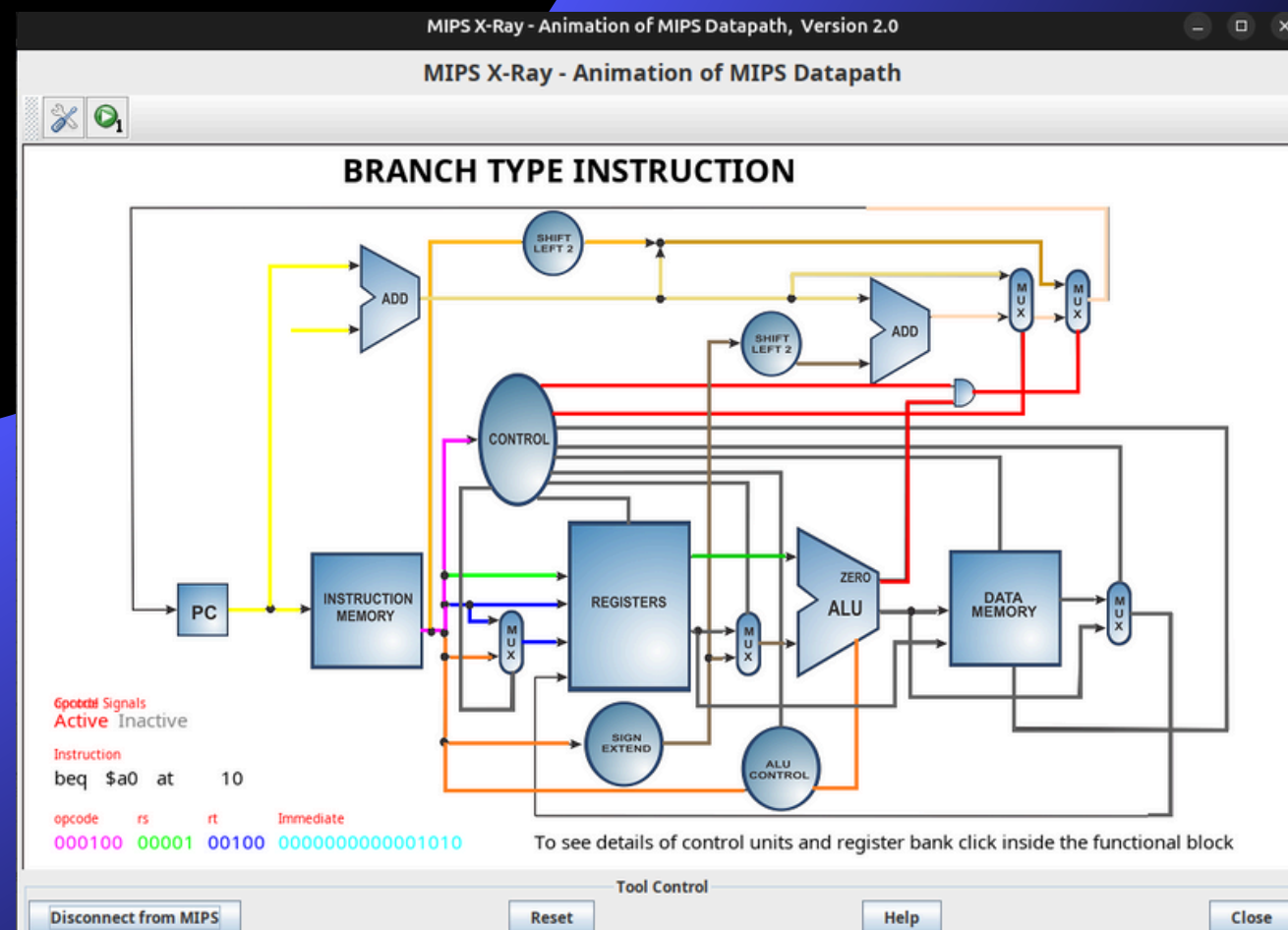
    add $v0, $a0, $v0 #$v0 recebe o valor dele mesmo somado com o valor retornado do anterior.

    jr $ra #"Jump Register" onde pula para o endereço de memoria daquele registrador e continua

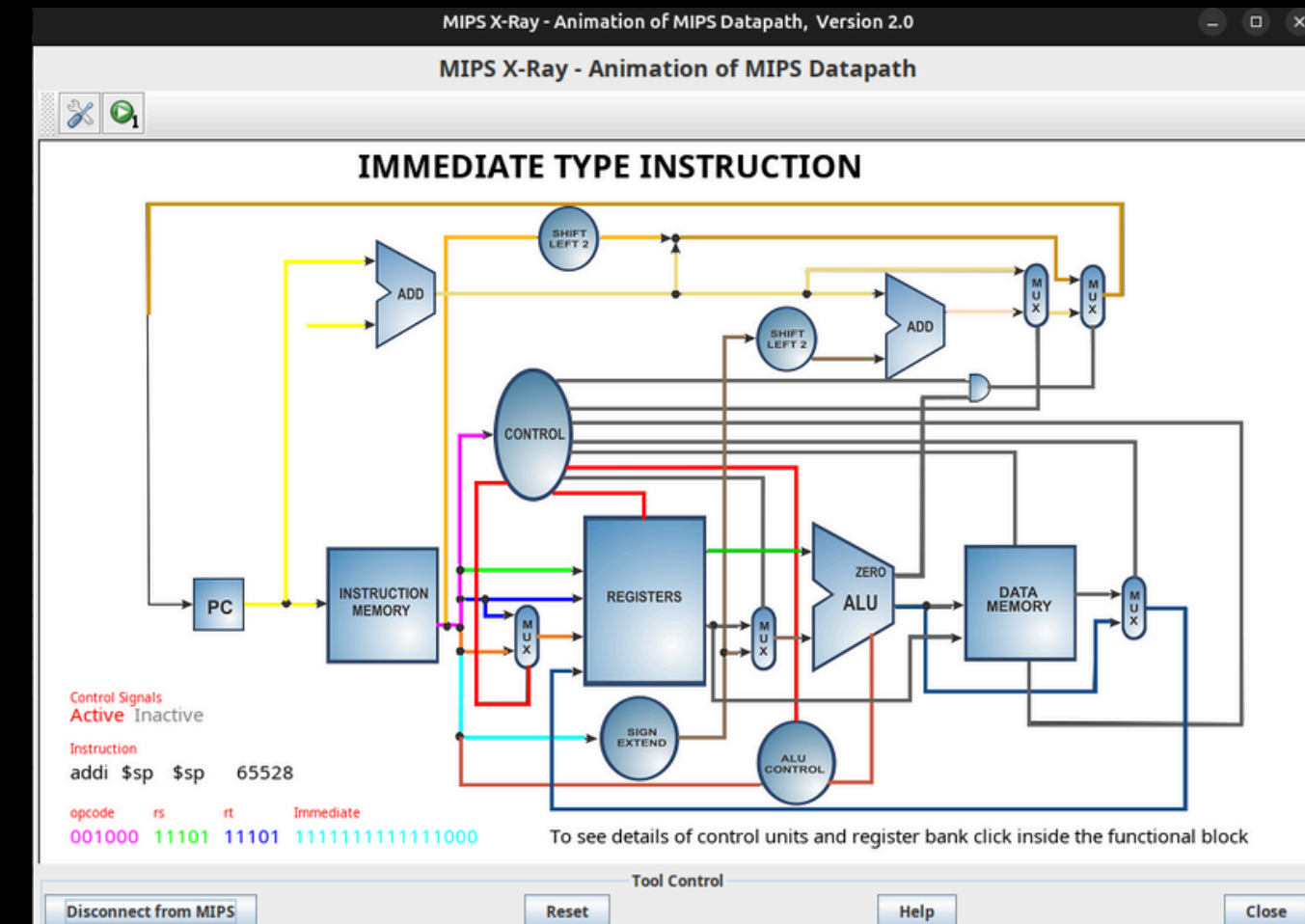
retorno:
    li $v0, 1 #Grava 1 no $v0. Dessa vez não é para imprimir um inteiro, é o valor 1 de fato que retorna no caso base
    jr $ra #"Jump Register" ele pula para o endereço de memória daquele registrador e continua
```

Essa é a função de somatório. Eu julguei como a mais importante pois é aqui onde acontece a recursão em si. Nos slides posteriores colocarei fotos do X-ray, especificadamente de uma parte da função.

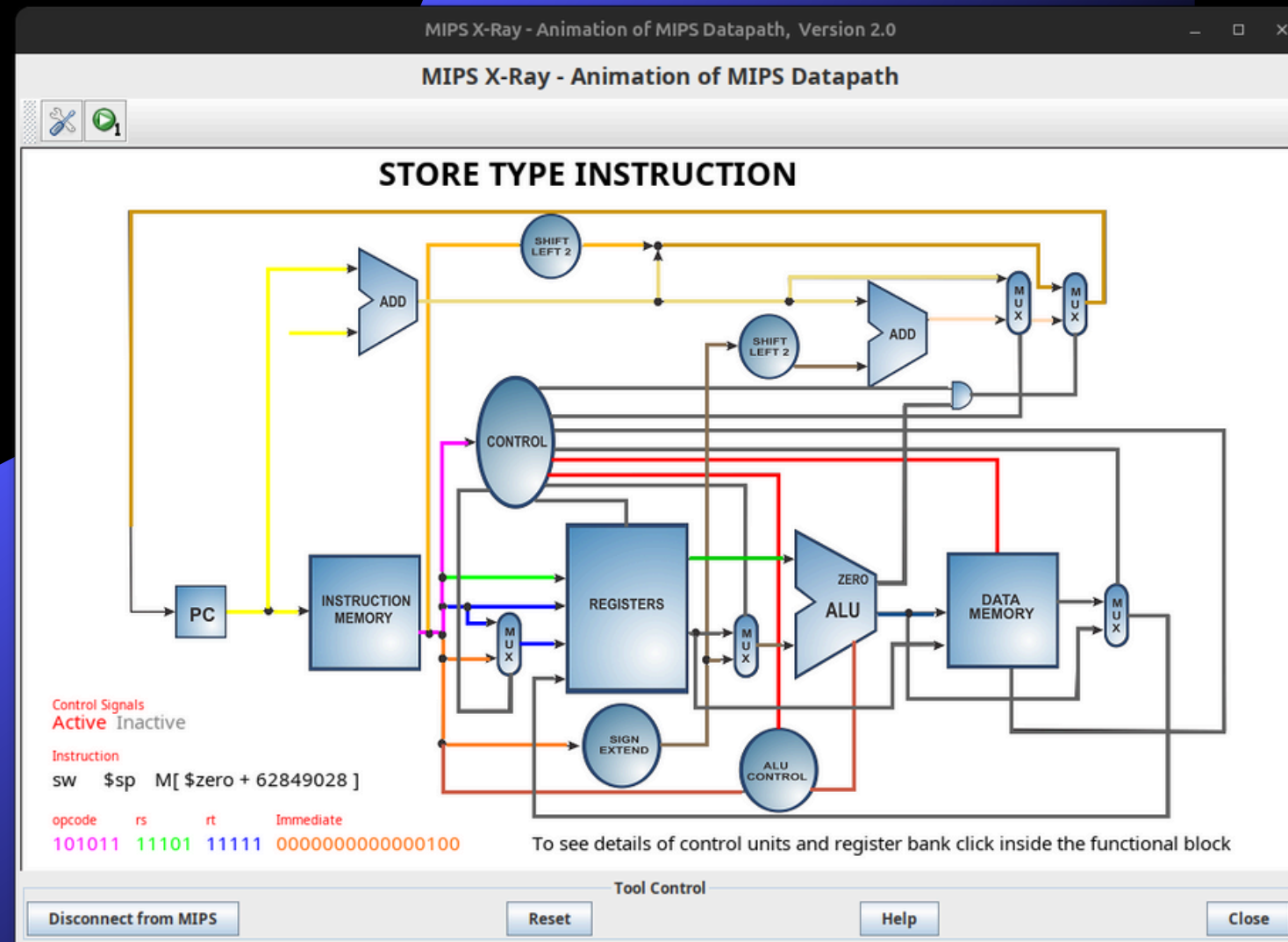
O motivo é que o resto da função seria a parte de retorno da recursividade e recuperação dos valores, e a parte que eu acho mais importante é a da chamada recursiva pois foi algo diferente para mim.



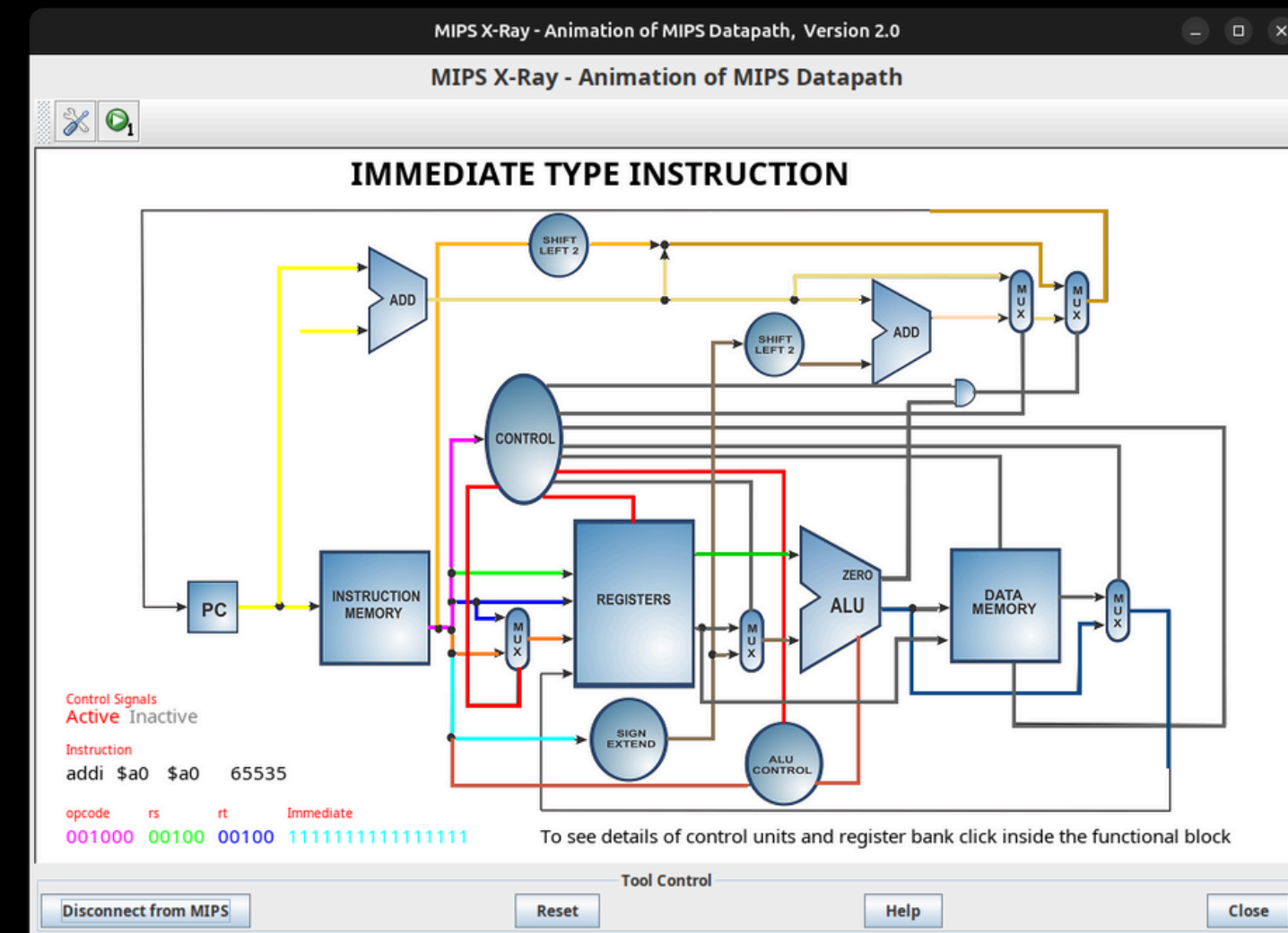
Instrução de branch if equal.



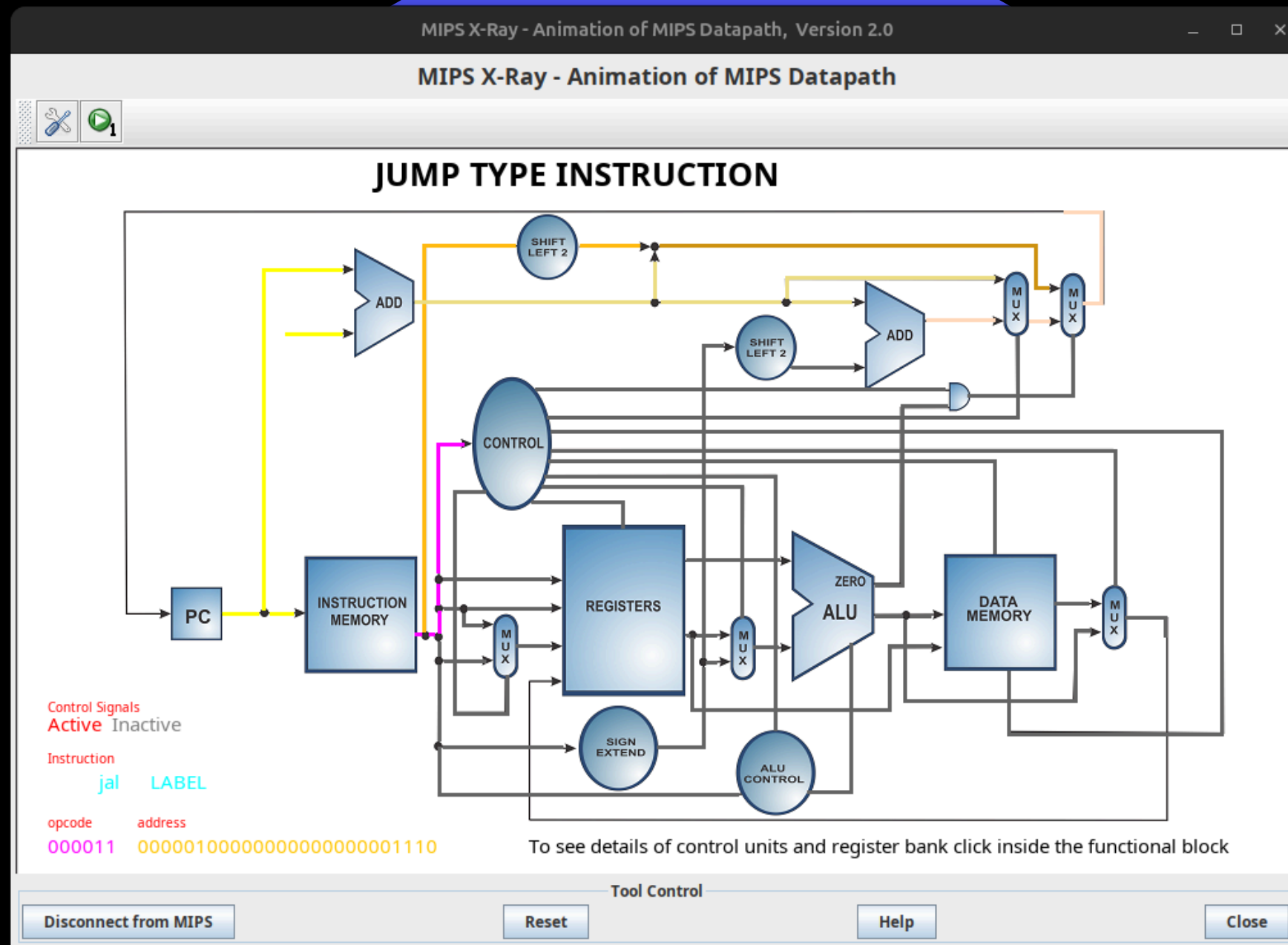
Instrução onde é reservado espaço na pilha. Linha 28



Nessa instrução o programa guardou naquele espaço reservado(deslocado de 4 bits) o valor do registrador \$ra. A função seguinte é exatamente a mesma mas sem deslocar 0 bits.



Instrução que soma o valor atual de N com -1, decrementando. Como será visto no código, é aqui onde o valor atual de N é preparado para ir para a próxima chamada de função. A parte do N-1 que tem no argumento da função.



Momento em que a instrução de jal chega na ALU. Note que apenas as linhas de cima estão pintadas. Isso acontece porque apenas os componentes que trabalham com o Program Counter vão ser ativados nesse caso.

Para mim foi sensacional descobrir como fazia essa recursão em assembly. Fazendo eu vi como era simples, mas parecia uma coisa muito complicada na minha cabeça e quando eu descobri, fazendo, que era assim simples eu fiquei rindo.



Esse é o momento em que pula para a parte do programa de volta pro começo da função. Por isso apenas os componentes que trabalham no PC vão ser afetados. Apenas o número que está em PC vai ser alterado.

```
53  retorno_imprimir_numeros:
54      jr $ra #Apenas pula para endereço salvo de retorno em $ra
55
56  imprimir_numeros:
57      bltz $t0, retorno_imprimir_numeros #"Branch low then zero" Se t0 for menor que 0, pule para essa função
58      li $v0, 1 #grava em v0 o código referente a impressão de um inteiro
59      move $a0, $t0 #move de t0 para a0 pois o sistema só lê do a0
60      syscall #Chama o sistema
61      li $v0, 11 #Mesma coisa para imprimir o \n
62      li $a0, 10 #Escreve o código de \n na tabela ascii em a0
63      syscall #imprime o \n
64      addi $t0, $t0, -1 #Soma o valor em t0 com -1 para imprimir o próximo
65      j imprimir_numeros #Pula para início da função novamente. Note que não precisa de link pois não preciso guardar nada da última iteração
66
```

Para a impressão dos números foi feita uma outra função. Algo interessante de se notar foi que um laço for, e uma função recursiva são basicamente a mesma coisa com o diferencial de que a recursiva salva as informações que ela precisa na pilha, como endereço do retorno e valor naquele momento. O for faz a mesma coisa, mas não salva nada da última iteração. Algo besta mas que só notei agora no assembly

Curiosidade: Enquanto fazia, o sistema imprimia na saída alguns valores "loucos" e após sair do Main voltava para o somatório. Foi então que eu percebi que faltava o encerramento do programa, o que não é necessário em C, a linguagem que foi traduzida. Uma pequena curiosidade de uma experiência simples mas para um programador iniciante em assembly achei interessante.

CONCLUSÕES FINAIS

O programa já era bem simples antes no C, em Assembly Mips só notei a otimização de que a linguagem ta conversando com a máquina mais direto do que a linguagem C, então é mais rápido. Mas fora isso eu não notei muita coisa pq o programa já é naturalmente bem simples

